Mod 2 — Lab 2: Classes + Inheritance (Foo, Animal, Dog, Cat)

What you're doing (and why)

In this lab you will write **three small classes** to practice core OOP ideas:

- A **class** is a blueprint for making objects (instances).
- **Instance variables** store per-object data (e.g., a pet's name).
- A **method** is a function that belongs to a class and uses `self`.
- **Inheritance** lets a child class reuse a parent class's code (`Dog` and `Cat` reuse `Animal`).
- `super()` is how a child class calls a parent method (especially the parent constructor).

You will also practice a light version of **test-driven development (TDD)**:

- Use the provided tests to clarify requirements and verify correctness.
- Work in a loop: write a little → run tests → fix → repeat.

What's provided

You are given two test files:

- `test_1foo.py` checks your `Foo` class.
- `test_2animals.py` checks your `Animal`, `Dog`, and `Cat` classes.

These files act like an automated "checklist."
They verify that your class names, method names, return values, and string formatting match the expected behavior. We will learn more about that next week, and you should be able to create your own tests.

Files you should have in lab2 folder

```
lab2/
  Foo.py
  Animals.py
  test_1foo.py
  test_2animals.py
```

How to run the tests

From inside the `lab2/` folder:

```
python test_1foo.py
python test_2animals.py
```

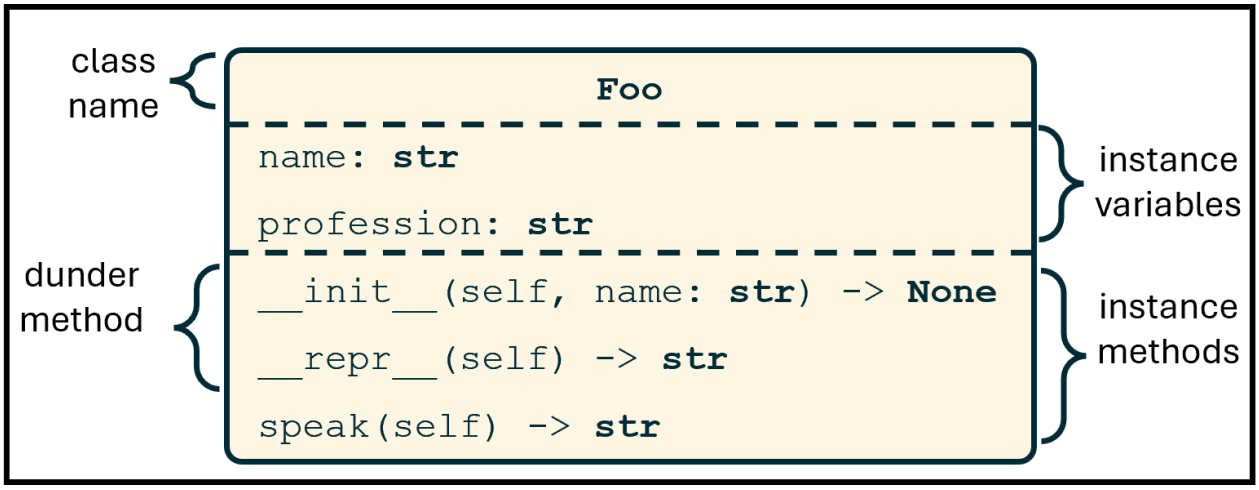
☑ **TDD workflow:** run tests early (they will fail), implement one requirement, run again, repeat until `OK`.

Part A — Build the `Foo` class (in `Foo.py`)

`Foo` is a small **class** used to practice:

- storing data in instance variables
- writing instance methods
- customizing object display with `__repr__`

The UML class diagram for Foo looks like this:



1) `__init__(self, name, profession)` *(constructor)*

What: Initializes a new `Foo` object and stores its data.
Why: Every object needs its own values (instance variables).

Requirements:

- Store:
  - `self.name`
  - `self.profession`

Example:

```
f = Foo("John", "Professor")
# f.name == "John"
# f.profession == "Professor"
```

☑ **Checkpoint:** run `python test_1foo.py` and see what has changed.

2) `speak(self) -> str` *(instance method)*

What: A method is a function attached to a class. It uses `self` to access the object's data.
Why: Methods define what objects *can do*.

Return **exactly**:

```
"<name> says hello!"
```

Example:

```
Foo("rachel", "Carpenter").speak()
# "rachel says hello!"
```

☑ **Checkpoint:** run `python test_1foo.py` and see what has changed.

3) `__repr__(self) -> str` *(string representation)*

What: Controls what Python shows when you print the object or inspect it.
Why: Makes debugging and output consistent (and tests check this).

Return **exactly**:

```
"Foo(<name>, <profession>)"
```

Example:

```
repr(Foo("John", "Professor"))
# "Foo(John, Professor)"
```

☑ **Checkpoint:** `python test_1foo.py` should ends with `OK`.

Part B — Build the `Animal`, `Dog`, and `Cat` classes (in `Animals.py`)

Here you practice **inheritance**:

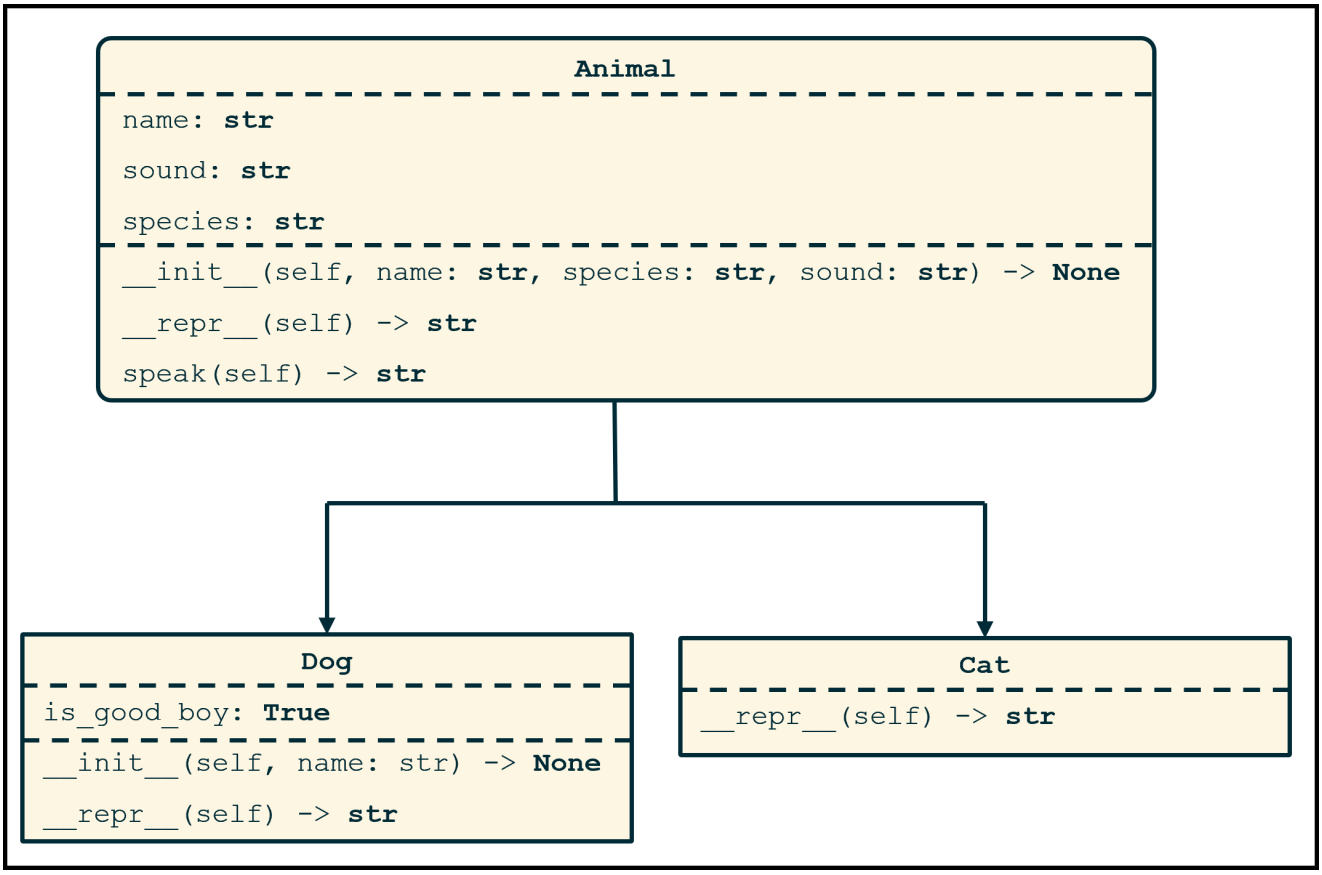
- `Animal` is the **parent/base** class.
- `Dog` and `Cat` are **child** classes that **inherit from** `Animal`.

Inheritance means:

- Child classes automatically get the parent's methods (unless overridden).
- You reuse code instead of rewriting it.

before we start writing the code, run `test_2animals.py`

Your code should reflect this class diagram:



1) Class `Animal` *(parent/base class)*

`Animal` defines the shared data and shared behavior.

`__init__(self, name, species="animal", sound="hi")`

What: Store an animal's name/species/sound as instance variables.
Why: These values are shared across all animals.

Store:

- `self.name`
- `self.species`

- `self.sound`

Example:

```
a1 = Animal("Arthur", "Ardvark")
# sound defaults to "hi"
```

`speak(self) -> str` *(instance method)*

What: Return a sentence describing the animal.
Why: Shows how a method can combine multiple instance variables.

Return **exactly**:

```
"<name>, a <species>, says <sound>!"
```

Example:

```
Animal("Arthur", "Ardvark").speak()
# "Arthur, a Ardvark, says hi!"
```

`__repr__(self) -> str`

What: Return a readable representation of an `Animal`.
Why: Helps debugging and is checked by the tests.

Return **exactly**:

```
"Animal(<name>, <species>, <sound>)"
```

Example:

```
repr(Animal("Colin", "Carpenter Bee", "buzz"))
# "Animal(Colin, Carpenter Bee, buzz)"
```

☑ **Checkpoint:** run `test_2animals.py`.

2) Class `Dog(Animal)` *(child class that inherits from Animal)*

A `Dog` **inherits** `Animal` behavior (like `speak`) but customizes initialization and representation.

`__init__(self, name, is_good_boy=True)`

What: Build a dog with fixed species/sound and one extra attribute.
Why: Demonstrates how a child class extends a parent class.

Dogs are always:

- `species == "dog"`
- `sound == "ruff"`

Using `super()`

`super()` lets a child call the parent class method. Here we use it to call the parent **constructor** so the parent sets `name`, `species`, and `sound` correctly.

Requirements:

1. Call the parent initializer:

- `super().__init__(name, "dog", "ruff")`

2. Add the dog-specific instance variable:
    - `self.is_good_boy = is_good_boy`

Example:

```
d = Dog("Arthur")
# d.name == "Arthur"
# d.species == "dog"
# d.sound == "ruff"
# d.is_good_boy == True
```

`__repr__(self) -> str`

What: Short display for dogs.
Why: Child classes can override a parent method.

Return **exactly**:

```
"Dog(<name>)"
```

Example:

```
repr(Dog("Colin"))
# "Dog(Colin)"
```

☑ **Checkpoint:** run `test_2animals.py`.

3) Class `Cat(Animal)` *(child class that inherits from Animal)*

A `Cat` also **inherits from** `Animal`.

Constructor

Do not write a `Cat.__init__`.
Why: If you don't define a constructor, Python uses the parent's (`Animal.__init__`) automatically. This is a great example of code reuse with inheritance.

So this should work:

```
c = Cat("Arthur", "cat", "meow")
# uses Animal.__init__ under the hood
```

`__repr__(self) -> str`

What: Custom display for cats.
Why: Another example of overriding a parent method.

Return **exactly**:

```
"Cat(<name>, <species>, <sound>)"
```

Example:

```
repr(Cat("Arthur", "cat", "meow"))
# "Cat(Arthur, cat, meow)"
```

Expected behavior (end-to-end example)

```
a1 = Animal("Arthur", "Ardvark")
d1 = Dog("Doug")
c1 = Cat("Caroline", "calico cat", "meow")

print(a1, d1, c1)
# Animal(Arthur, Ardvark, hi) Dog(Doug) Cat(Caroline, calico cat, meow)

print(a1.speak(), d1.speak(), c1.speak())
# Arthur, a Ardvark, says hi! Doug, a dog, says ruff! Caroline, a calico cat, s
```

☑ **Checkpoint B:** `python test_2animals.py` ends with `OK`.

Common mistakes (fast checks)

- **Printing instead of returning:** these methods must **return strings**.
- **Tiny formatting differences:** tests check punctuation/spaces exactly (commas, spaces, `!`).
- **Forgetting inheritance:** `Dog` and `Cat` must be defined as `Dog(Animal)` and `Cat(Animal)`.
- **Not using `super()` in `Dog`:** use it to reuse `Animal` initialization cleanly.

Before you submit

1. Run both tests and confirm `OK`. Do **not** submit the test files.
2. Submit:
    - `Foo.py`
    - `Animals.py`
3. Back up your work (OneDrive/Git/etc.).